



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE204L- Design and Analysis of Algorithms

Dr. Iyappan Perumal

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT,Vellore.



Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms
2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively
3. To synthesize efficient algorithms in various engineering design situations



BCSE204L- Design and Analysis of Algorithms-

Course outcome

1. Apply the mathematical tools to analyse and derive the running time of the algorithms
2. Demonstrate the major algorithm design paradigms.
3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.
4. Articulating Randomized Algorithms.
5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.



BCSE204L- Design and Analysis of Algorithms- Syllabus

Module:1	Design Paradigms: Greedy, Divide and Conquer Techniques	6 hours
Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - Greedy techniques: Fractional Knapsack Problem, and Huffman coding - <u>Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm.</u>		
Module:2	Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques	10 hours
Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP- Backtracking: N-Queens problem, Subset Sum, Graph Coloring- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem		
Module:3	String Matching Algorithms	5 hours
Naïve String-matching Algorithms, KMP algorithm, Rabin-Karp Algorithm, Suffix Trees.		
Module:4	Graph Algorithms	6 hours
All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm - Network Flows: Flow Networks, Maximum Flows: Ford-Fulkerson, Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to maximum matching problem		
Module:5	Geometric Algorithms	4 hours
Line Segments: Properties, Intersection, sweeping lines - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.		
Module:6	Randomized algorithms	5 hours
Randomized quick sort - The hiring problem - Finding the global Minimum Cut.		
Module:7	Classes of Complexity and Approximation Algorithms	7 hours
The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman		
Module:8	Contemporary Issues	2 hours

BCSE204L- Design and Analysis of Algorithms- References

Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3rd Edition, MIT Press.

Reference Books:

1. Jon Kleinberg, Éva Tardos ,Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press, 1995 (Online Print – 2013)
3. Ravindra K.Ahuja, Thomas L.Magnanti and James B. Orlin, “ Network Flows: Theory, Algorithms and Applications”, Pearson Education, 2014.



Module I: Design Paradigms: Greedy, Divide and Conquer Techniques(6 Hours)

- **Overview and Importance of Algorithms**
 - **Stages of algorithm development:**
 - Describing the problem
 - Identifying a suitable technique
 - Design of an algorithm
 - Derive Time Complexity,
 - Proof of Correctness of the algorithm
- **Illustration of Design Stages**
 - **Greedy techniques:**
 - Fractional Knapsack Problem
 - Huffman coding
 - **Divide and Conquer:**
 - **Maximum Subarray**
 - Karatsuba faster integer multiplication algorithm.



Divide and Conquer

- **Divide** the problem into a number of sub problems that are smaller instances of the same problem.
- **Conquer** the sub problems by solving them recursively. If the sub problem sizes are small enough, however, just solve the sub problems in a straightforward manner.
- **Combine** the solutions to the sub problems into the solution for the original problem.
- Example: **Binary Search, Merge Sort, Quick Sort**



Divide and Conquer

- Partitions the problem into disjoint sub problems, solve the sub problems recursively, and then combine their solutions to solve the original problem.
- Given a function to compute on n inputs the divide-and-conquer strategy suggests splitting the inputs into k distinct subsets, $1 < k < n$, yielding k sub problems
- Solve the k sub problems and find a method to combine the sub-solutions to a solution of the whole problem. If sub problems are still relatively large, then the divide-and-conquer strategy can possibly be reapplied.



Divide and Conquer Strategy

Algorithm DAndC(P)

```
{  
    if Small( $P$ ) then return  $S(P)$ ;  
    else  
    {  
        divide  $P$  into smaller instances  $P_1, P_2, \dots, P_k$ ,  $k \geq 1$ ;  
        Apply DAndC to each of these subproblems;  
        return Combine(DAndC( $P_1$ ), DAndC( $P_2$ ), ..., DAndC( $P_k$ ));  
    }  
}
```

Recurrence relation of the Divide and Conquer Strategy

$$T(n) = \begin{cases} g(n) & n \text{ small} \\ T(n_1) + T(n_2) + \dots + T(n_k) + f(n) & \text{otherwise} \end{cases}$$



Maximum-Subarray Problem

- **Problem Statement:**
- **Input:** An array $A[1...n]$ of (positive/negative) numbers.
- **Output:** Finding the sum of contiguous subarray of numbers which has the largest sum.



Maximum-Subarray Problem- Approaches

- Brute Force
- Divide and Conquer
- Greedy Algorithm
- Dynamic Programming-Kadane's Algorithm



Maximum-Subarray Problem- Brute Force Approach

0	1	2	3
-1	3	4	-2

Sub Array	Sum
-1	-1
-1 3	2
-1 3 4	6
-1 3 4 -2	4
3	3
3 4	7
3 4 -2	5
4	4
4 -2	2
-2	-2

Maximum-Subarray Problem- Brute Force Approach



maxSum

7

sum

-2

$O(n^2)$

```

maxSum = 0
for( i=0 to len(a) )
{
    sum = 0
    for( j=i to len(a) )
        sum = sum+a[j]
        maxSum = max(sum,maxSum)
}
    
```

Maximum-Subarray Problem- Brute Force Approach

- **Brute Force Approach** runs in two loops.
- The outer loop picks the beginning element, the inner loop finds the maximum possible sum with first element picked by outer loop and compares this maximum with the overall maximum.
- Finally, return the overall maximum.
- The time complexity of the **Brute Force** method is $O(n^2)$.



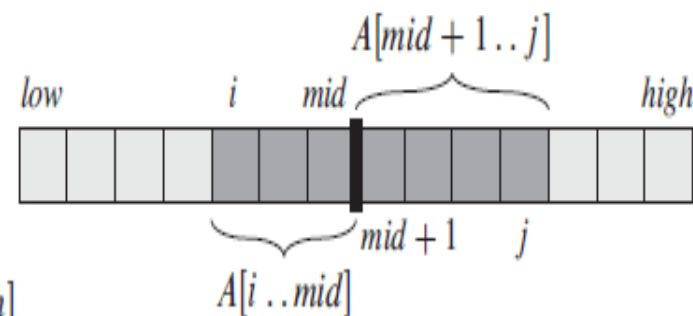
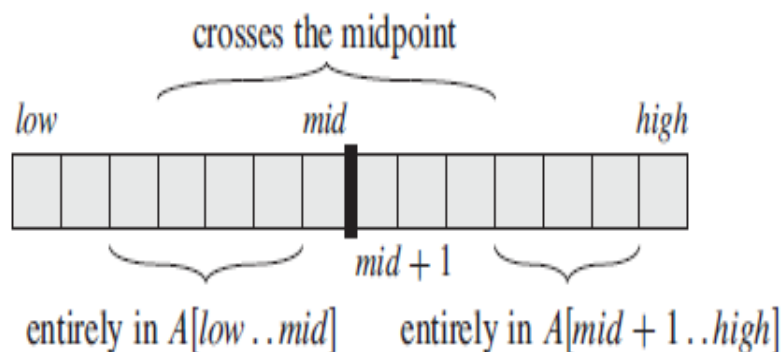
Maximum-Subarray Problem- Divide and Conquer Approach

- Divide the given array in two halves
- Return the maximum of following three
 - Maximum subarray sum in left half (Make a recursive call)
 - Maximum subarray sum in right half (Make a recursive call)
 - Maximum subarray sum such that the subarray crosses the midpoint



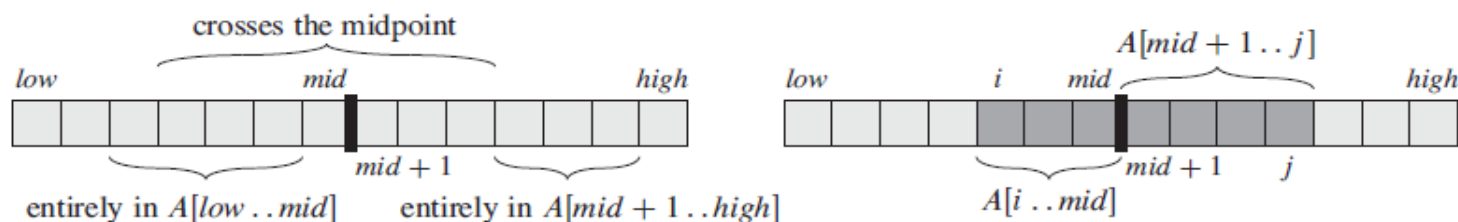
Maximum-Subarray Problem- Divide and Conquer Approach

- Divide-and-conquer suggests that we divide the subarray into two subarrays of as equal size as possible.
- That is, we find the midpoint, say mid , of the subarray, and consider the subarrays $A[low.. mid]$ and $A[mid.. High]$



Maximum-Subarray Problem- Divide and Conquer Approach

- As shown in the figure, any contiguous subarray $A[i..j]$ of $A[\text{low}.. \text{High}]$ must lie exactly in one of the following places:



- entirely in the subarray $A[\text{low}.. \text{mid}]$,
so that **$\text{low} \leq i \leq j \leq \text{mid}$** ,
 - entirely in the subarray $A[\text{mid}+1.. \text{high}]$,
so that **$\text{mid} < i \leq j \leq \text{high}$** , or
 - crossing the midpoint,
so that **$\text{low} \leq i \leq \text{mid} < j \leq \text{high}$** .
- Therefore, a maximum subarray of $A[\text{low}... \text{high}]$ must lie in exactly one of these places

Maximum-Subarray Problem- Divide and Conquer Approach- Demonstration

Given an integer array `nums`, find the contiguous subarray (containing at least one number) which has the largest sum and return its sum.

Example:

Input: `[-2,1,-3,4,-1,2,1,-5,4]`,

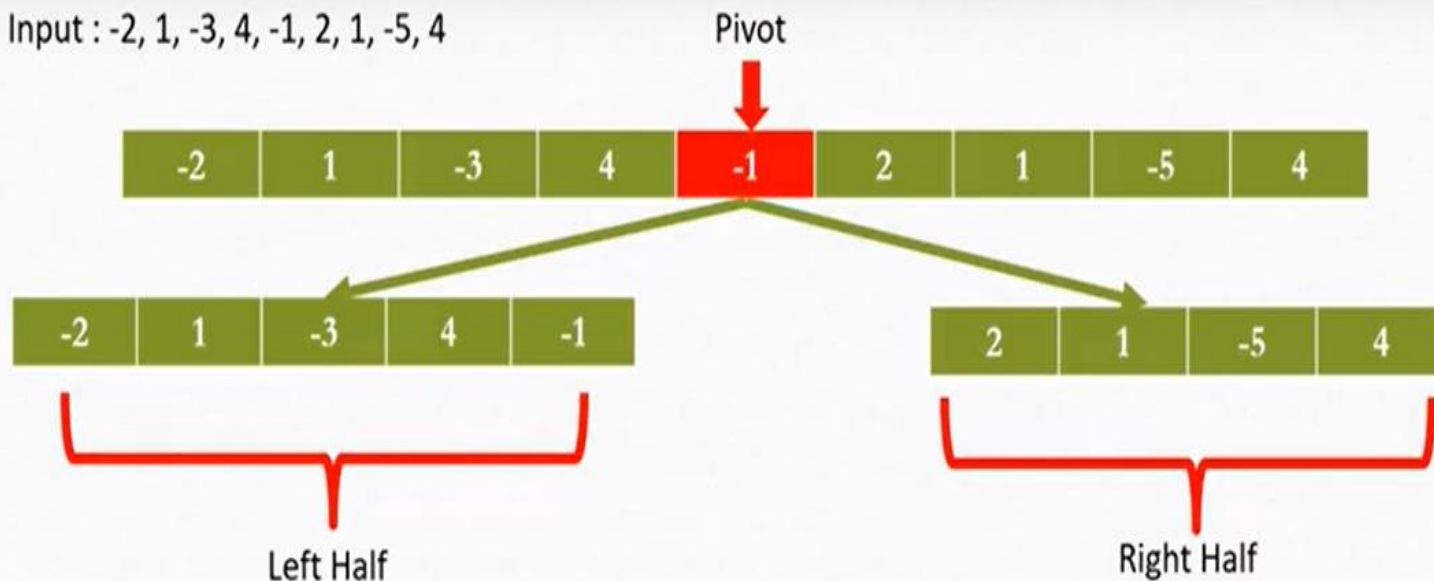
Output: 6

Explanation: `[4,-1,2,1]` has the largest sum = 6.



Maximum-Subarray Problem- Divide and Conquer Approach- Demonstration

Input : -2, 1, -3, 4, -1, 2, 1, -5, 4



$$\text{Index of Pivot} = \frac{(\text{begin} + \text{end})}{2} = \frac{(0+8)}{2} = 4$$

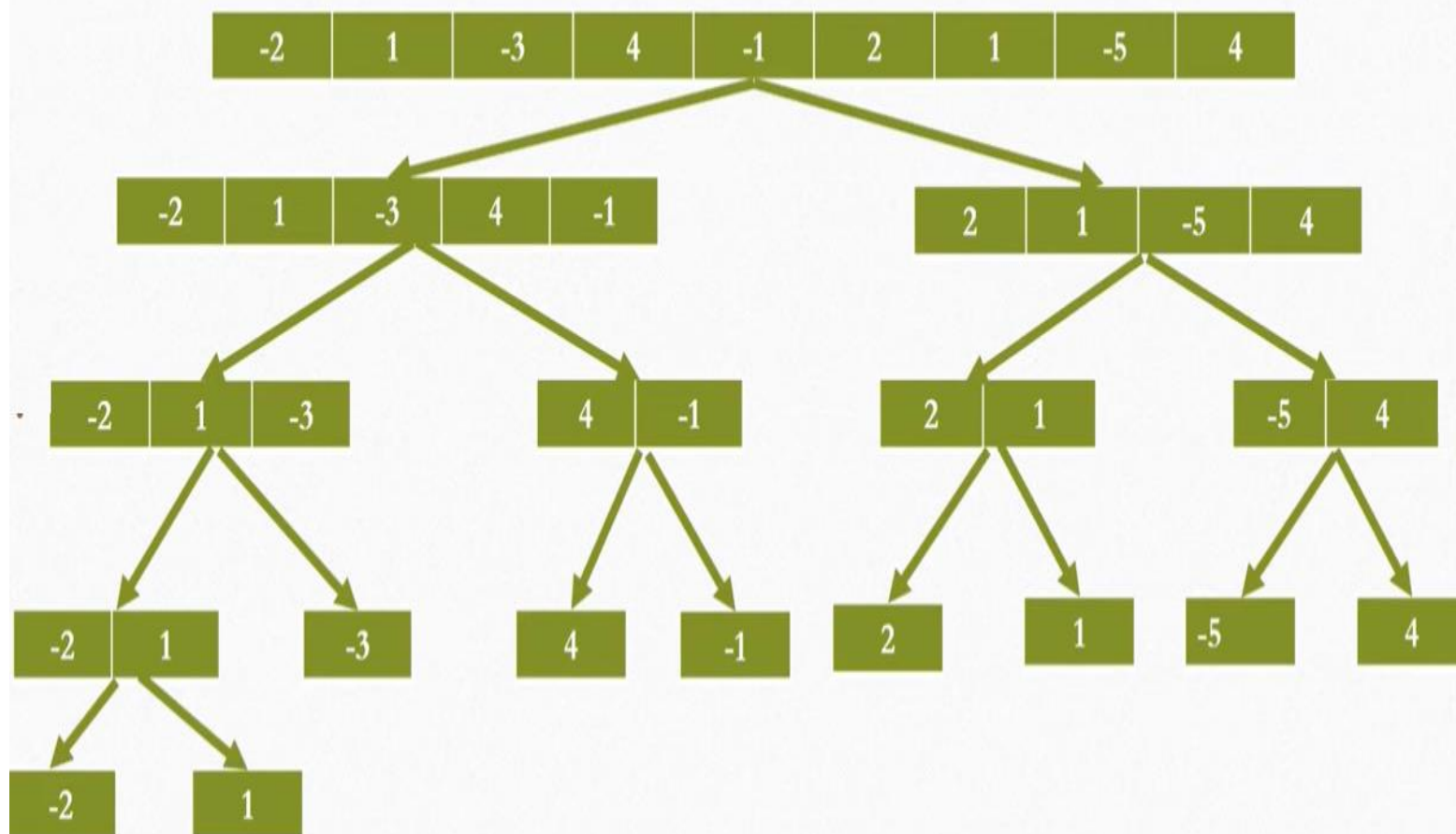
Left Half = begin to pivot (inclusive)

Right Half = pivot +1 to end



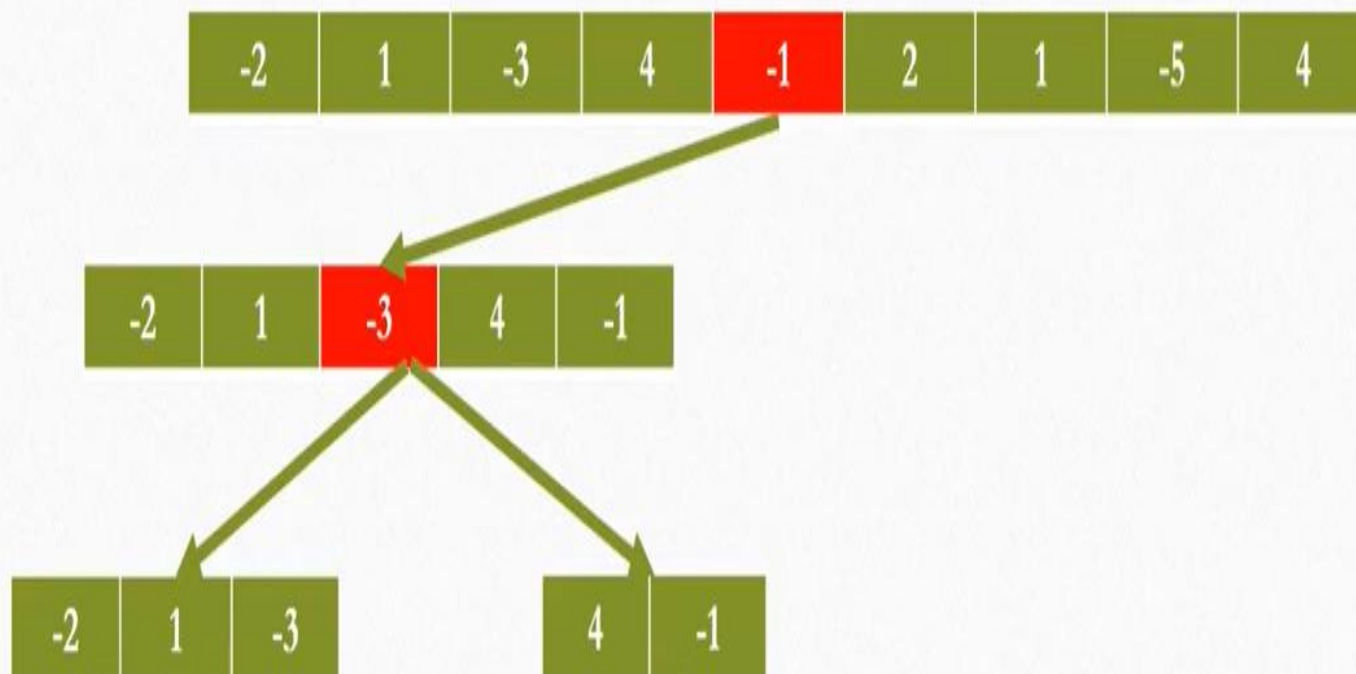
Maximum-Subarray Problem- Divide and Conquer Approach- Demonstration

Input : -2, 1, -3, 4, -1, 2, 1, -5, 4



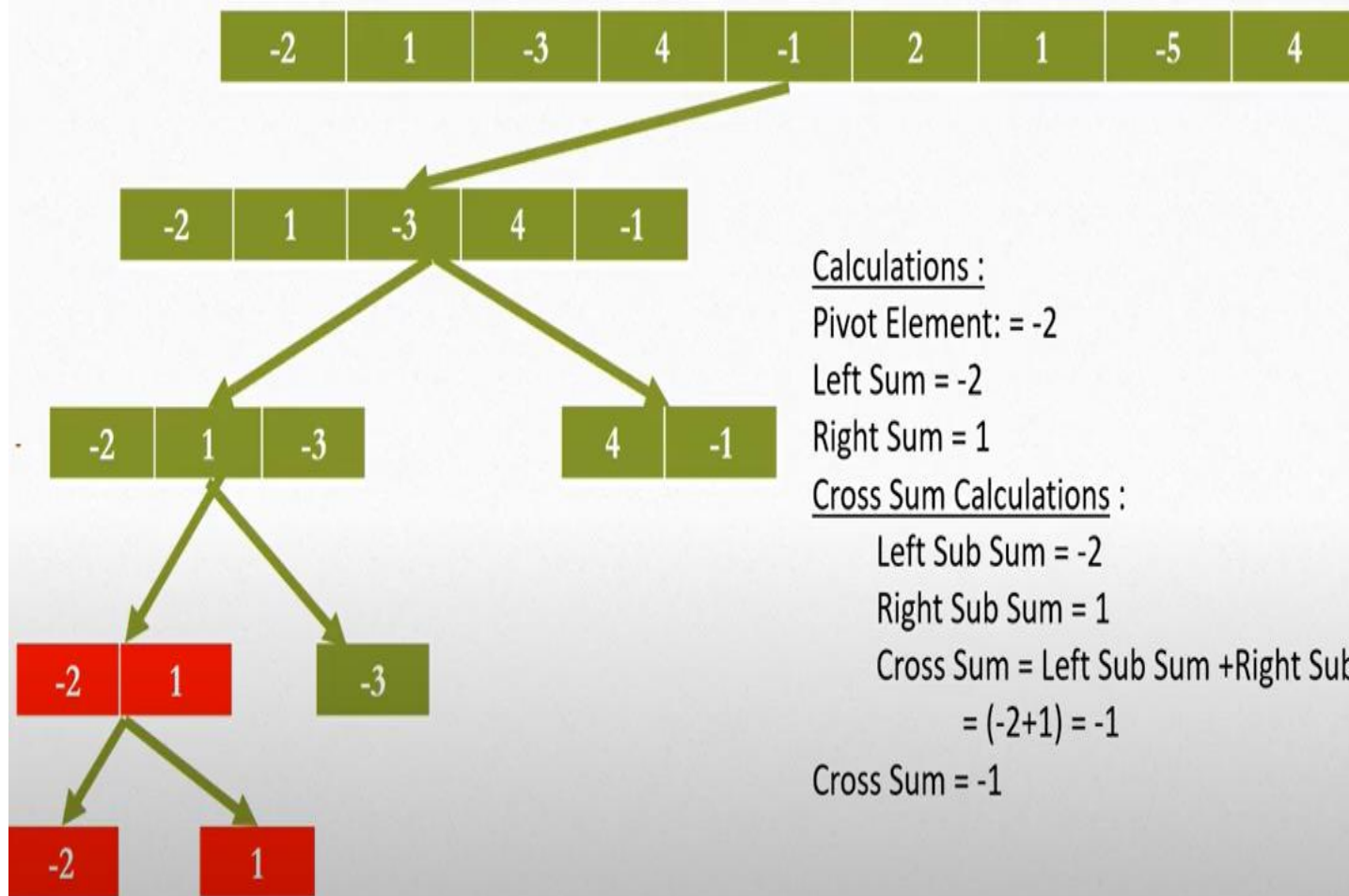
Maximum-Subarray Problem- Divide and Conquer Approach- Demonstration

Input : -2, 1, -3, 4, -1, 2, 1, -5, 4



Maximum-Subarray Problem- Divide and Conquer Approach- Demonstration

Input : -2, 1, -3, 4, -1, 2, 1, -5, 4



Calculations :

Pivot Element: = -2

Left Sum = -2

Right Sum = 1

Cross Sum Calculations :

Left Sub Sum = -2

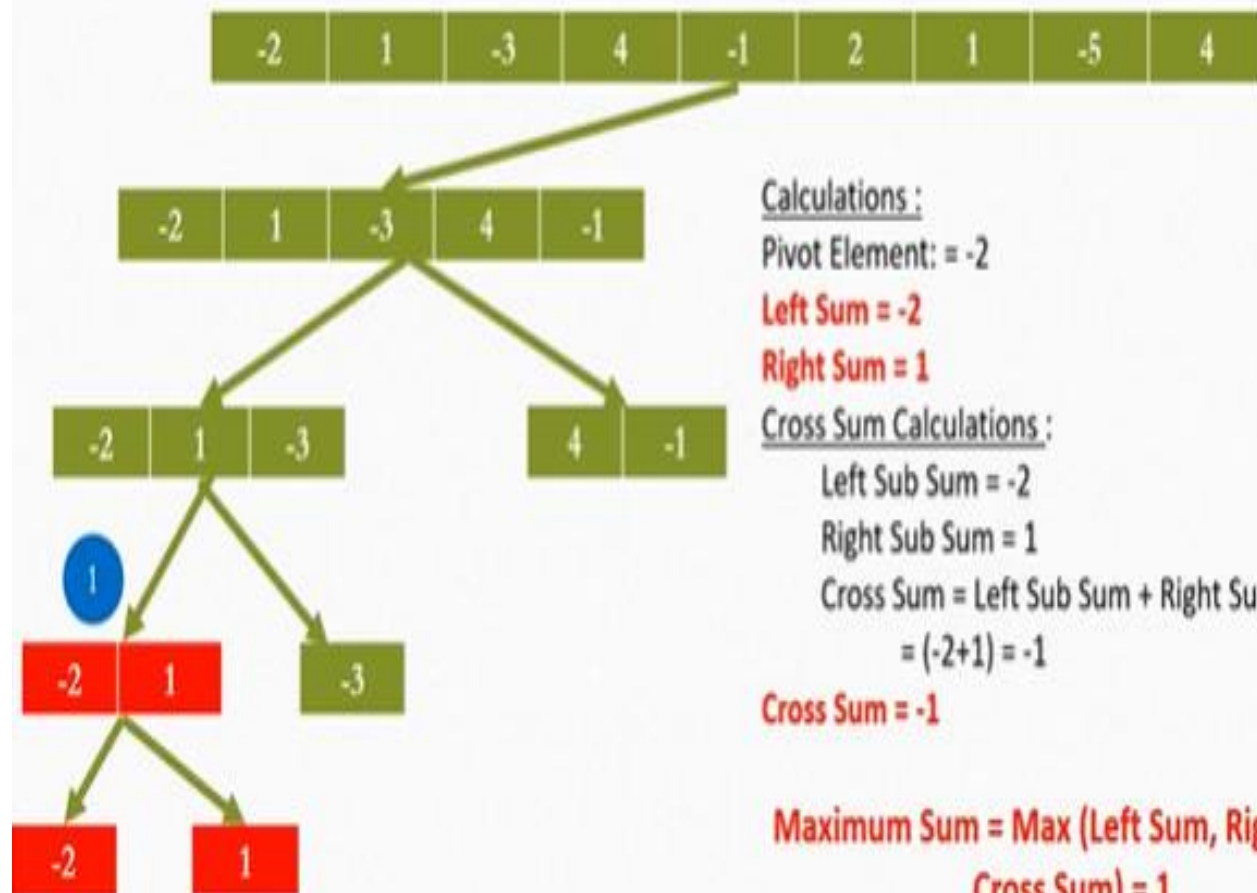
Right Sub Sum = 1

Cross Sum = Left Sub Sum + Right Sub Sum
= $(-2+1) = -1$

Cross Sum = -1

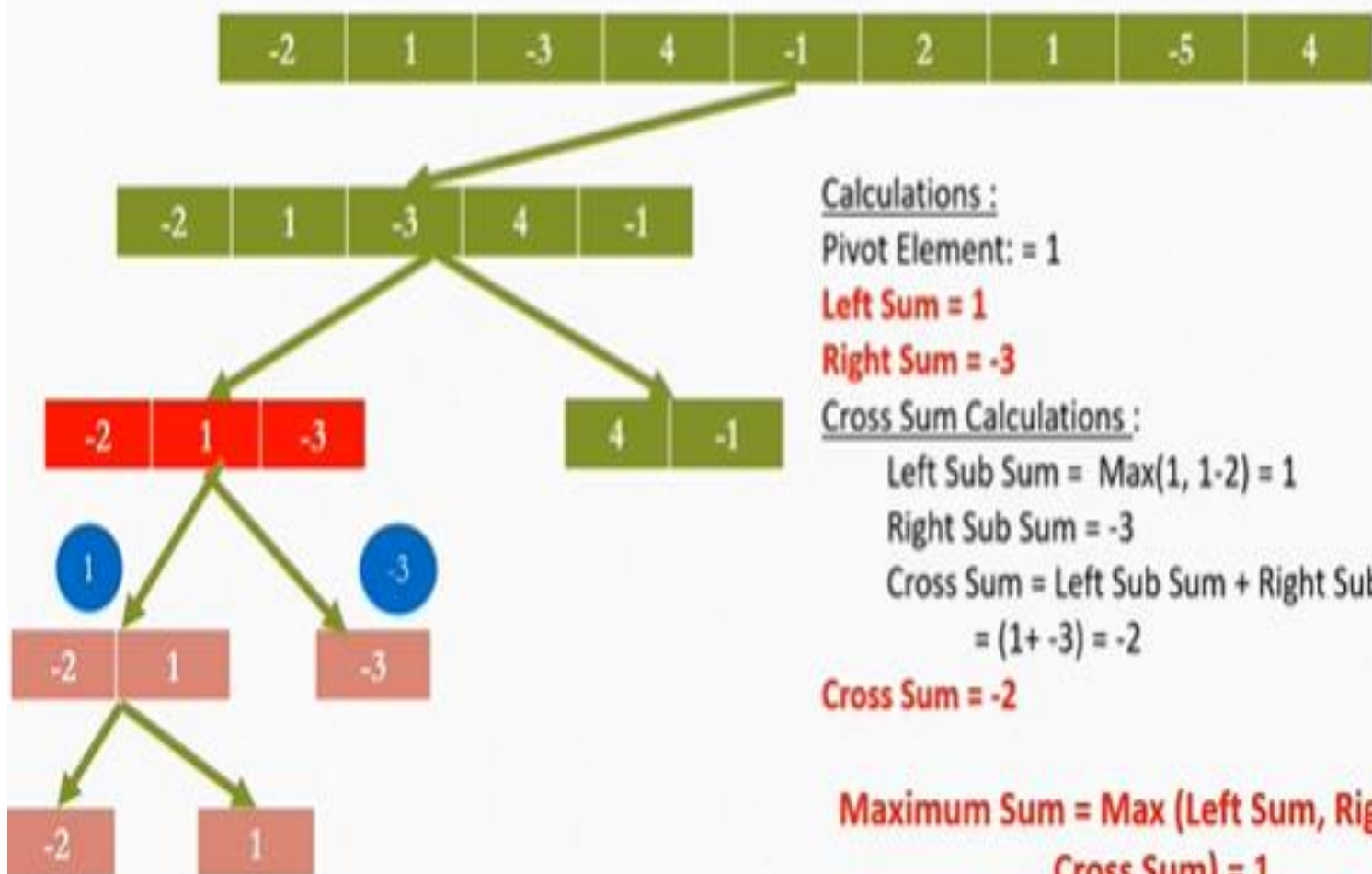
Maximum-Subarray Problem- Divide and Conquer Approach- Demonstration

Input : -2, 1, -3, 4, -1, 2, 1, -5, 4



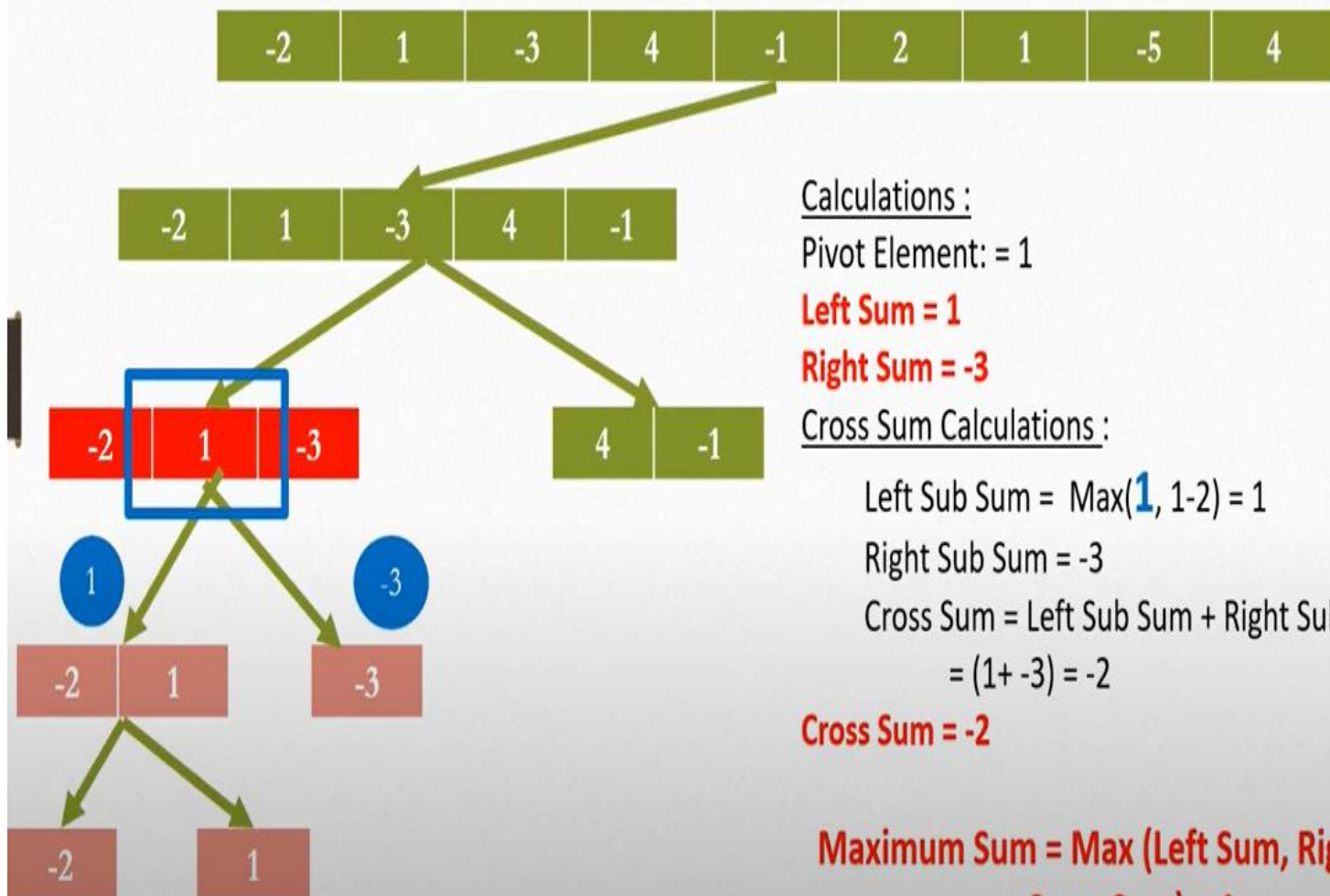
Maximum-Subarray Problem- Divide and Conquer Approach- Demonstration

Input : -2, 1, -3, 4, -1, 2, 1, -5, 4



Maximum-Subarray Problem- Divide and Conquer Approach- Demonstration

Input : -2, 1, -3, 4, -1, 2, 1, -5, 4



Calculations :

Pivot Element: = 1

Left Sum = 1

Right Sum = -3

Cross Sum Calculations :

Left Sub Sum = $\text{Max}(1, 1-2) = 1$

Right Sub Sum = -3

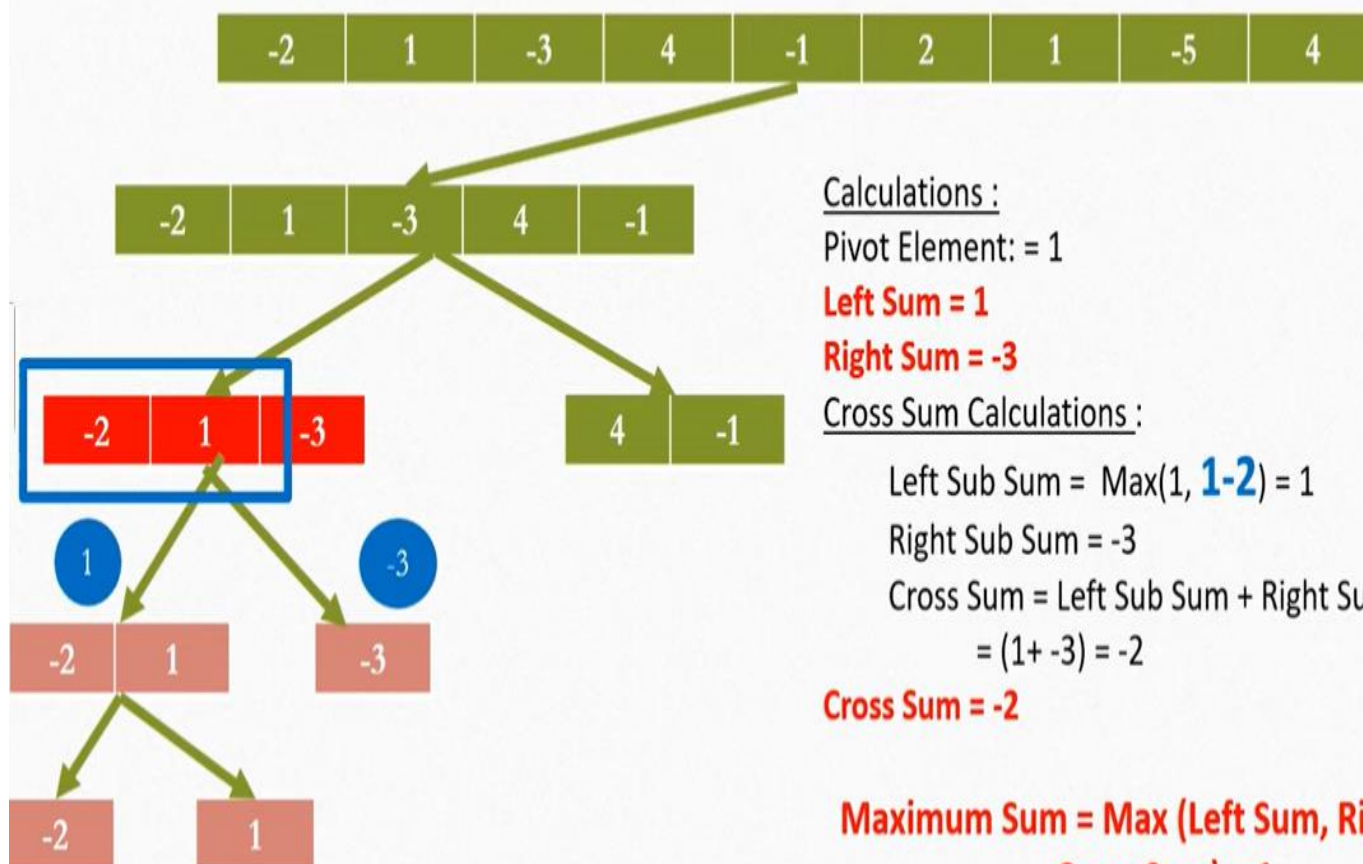
Cross Sum = Left Sub Sum + Right Sub Sum
= $(1 + -3) = -2$

Cross Sum = -2

**Maximum Sum = Max (Left Sum, Right Sum
Cross Sum) = 1**

Maximum-Subarray Problem- Divide and Conquer Approach- Demonstration

Input : -2, 1, -3, 4, -1, 2, 1, -5, 4



Calculations :

Pivot Element: = 1

Left Sum = 1

Right Sum = -3

Cross Sum Calculations :

Left Sub Sum = $\text{Max}(1, \mathbf{1-2}) = 1$

Right Sub Sum = -3

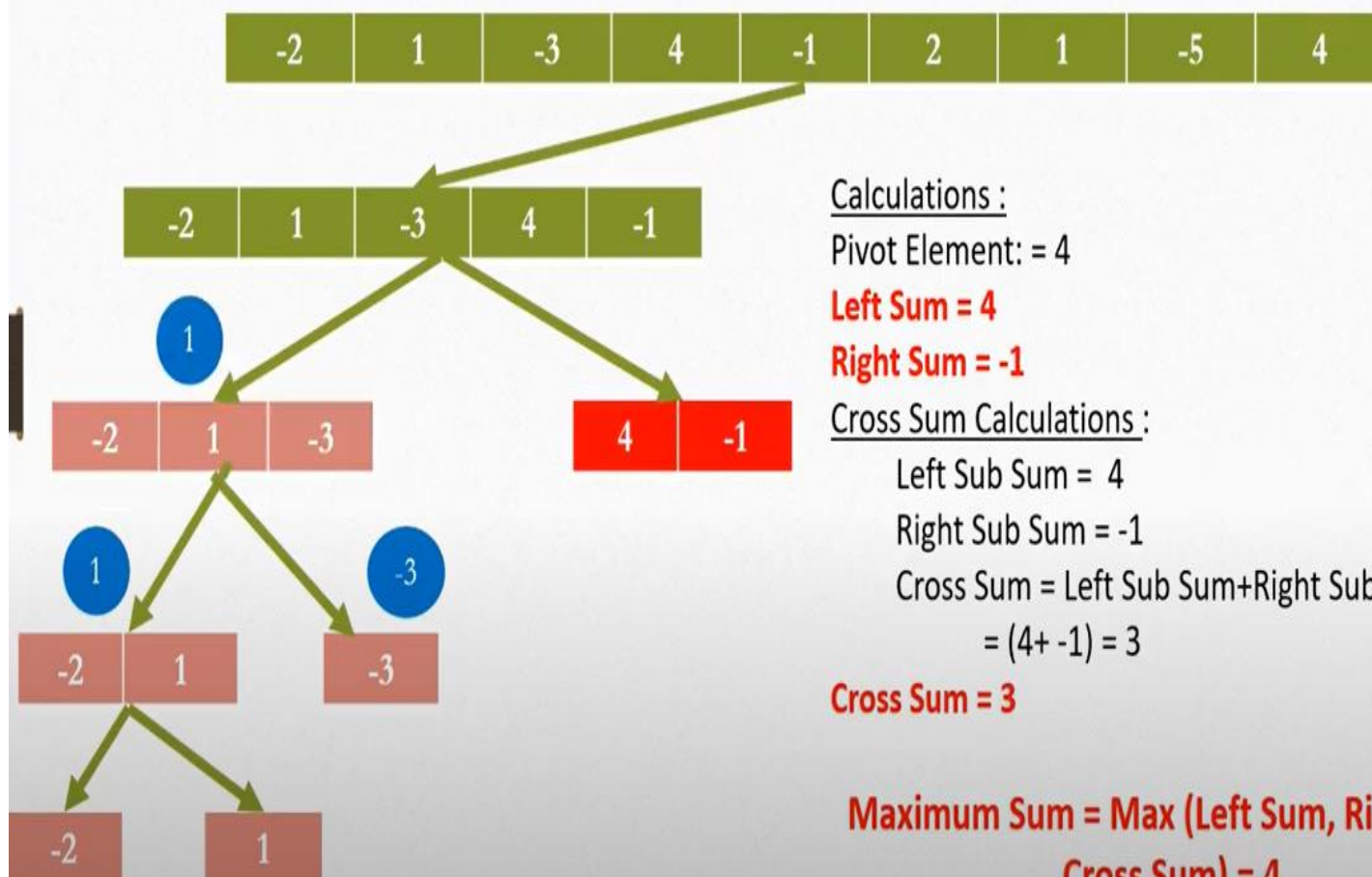
Cross Sum = Left Sub Sum + Right Sub Sum
= $(1 + -3) = -2$

Cross Sum = -2

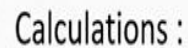
**Maximum Sum = Max (Left Sum, Right Sum
Cross Sum) = 1**

Maximum-Subarray Problem- Divide and Conquer Approach- Demonstration

Input : -2, 1, -3, 4, -1, 2, 1, -5, 4



Input : -2, 1, -3, 4, -1, 2, 1, -5, 4



Pivot Element: = -3

Left Sum = 1

Right Sum = 4

Cross Sum Calculations :

Left Sub Sum = **Max(-3, -3+1, -3+1-2) = -2**

Right Sub Sum = $\text{Max}(4, 4-1) = 4$

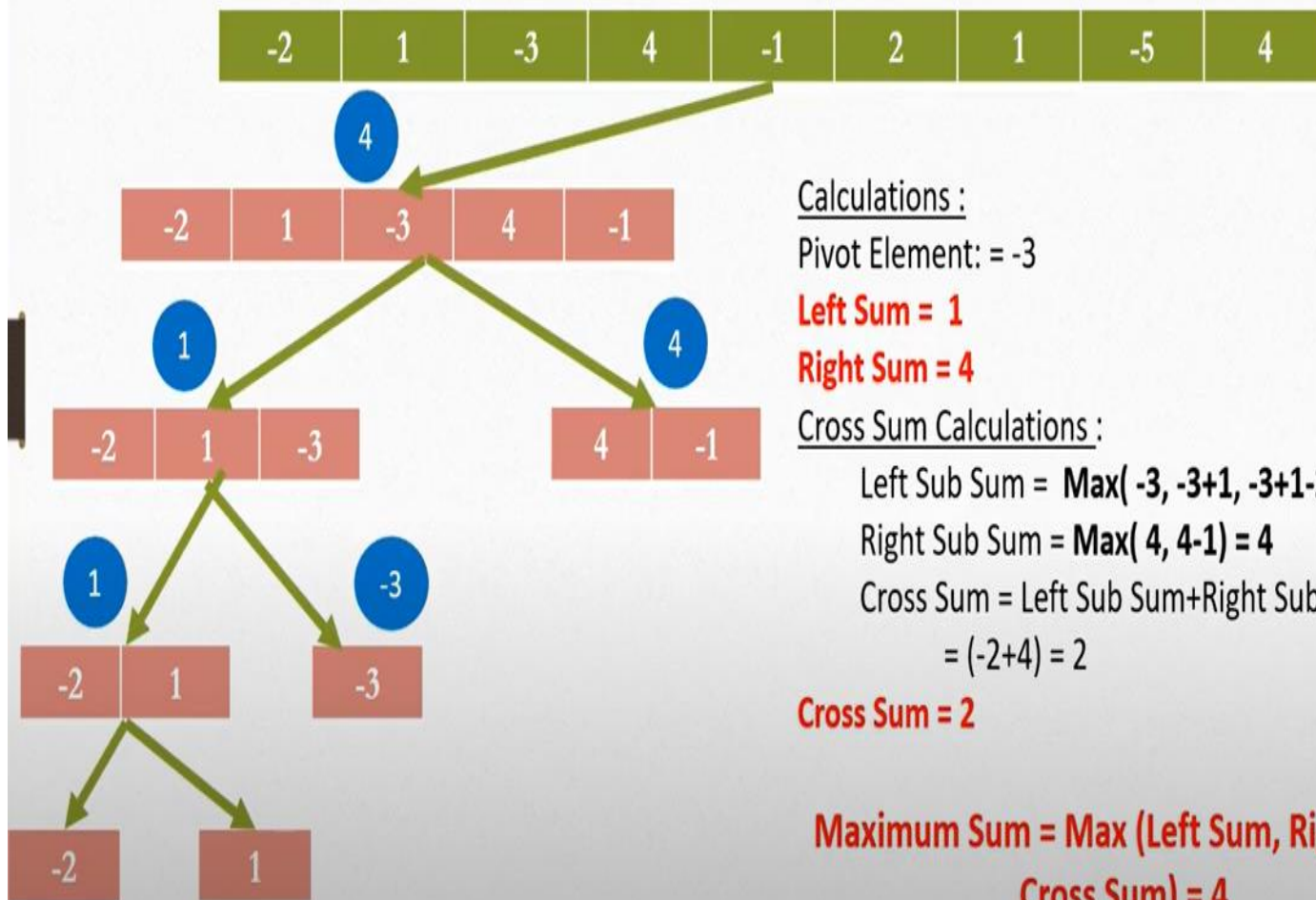
$$\begin{aligned}\text{Cross Sum} &= \text{Left Sub Sum} + \text{Right Sub Sum} \\ &= (-2+4) = 2\end{aligned}$$

Cross Sum = 2

**Maximum Sum = Max (Left Sum, Right Sum
Cross Sum) = 4**

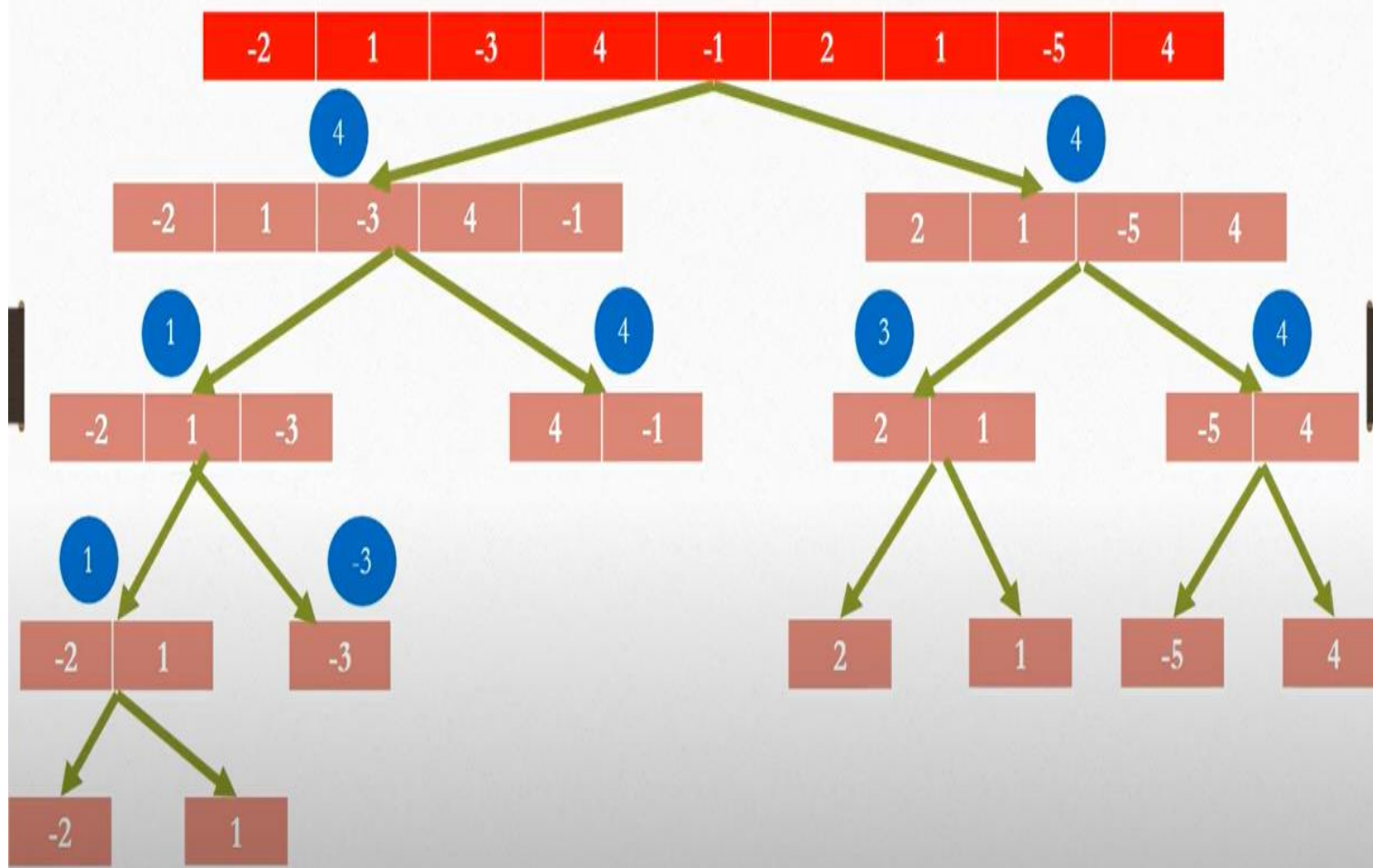
Maximum-Subarray Problem- Divide and Conquer Approach- Demonstration

Input : -2, 1, -3, 4, -1, 2, 1, -5, 4



Maximum-Subarray Problem- Divide and Conquer Approach- Demonstration

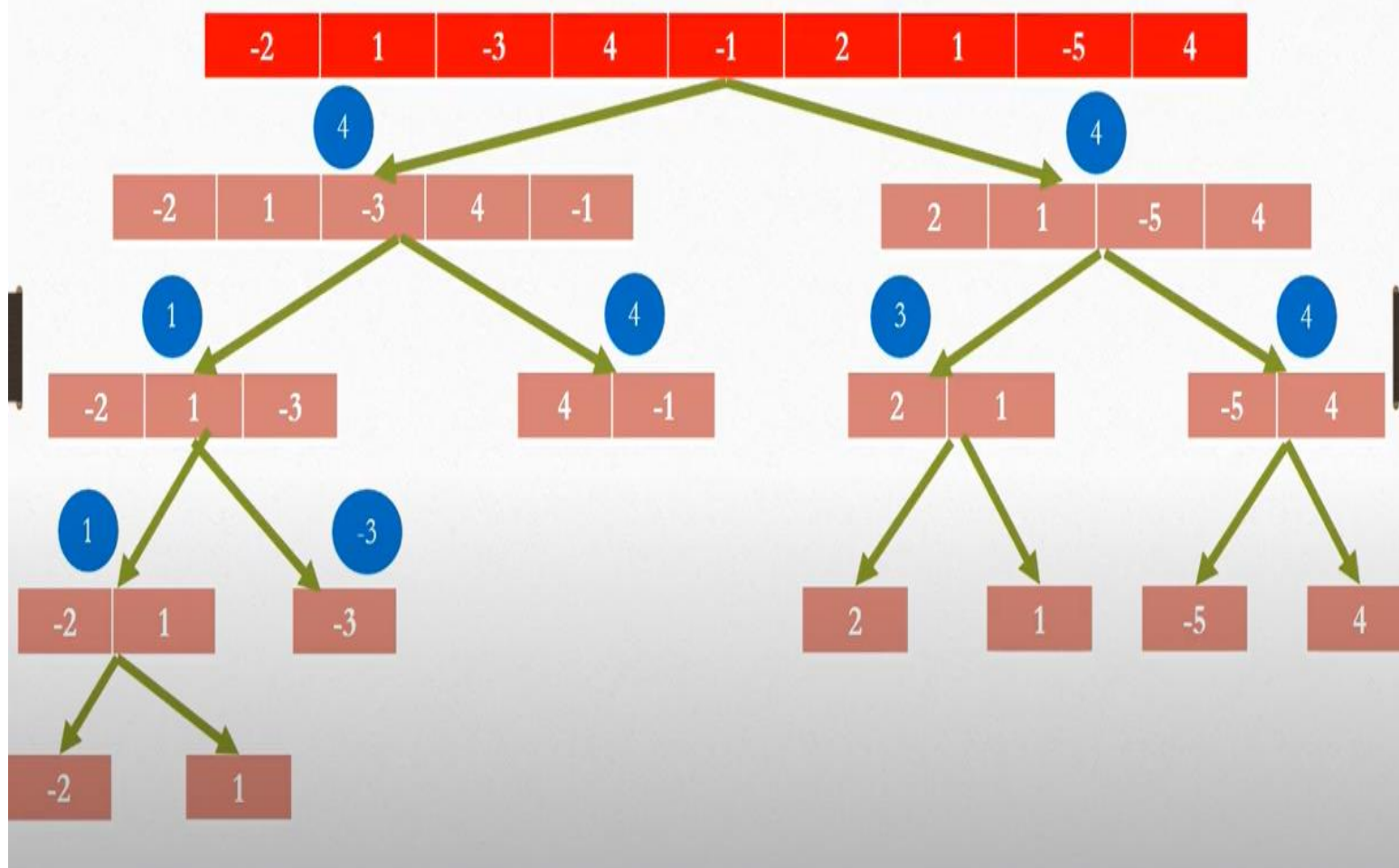
Input : -2, 1, -3, 4, -1, 2, 1, -5, 4



Maximum-Subarray Problem- Divide and Conquer Approach- Demonstration

Input : -2, 1, -3, 4, -1, 2, 1, -5, 4

Left Sum = 4, Right Sum = 4,
Cross Sum = ??



Maximum-Subarray Problem- Divide and Conquer Approach- Demonstration

Input : -2, 1, -3, 4, -1, 2, 1, -5, 4

Left Sum = 4, Right Sum = 4,
Cross Sum = ??

-2	1	-3	4	-1	2	1	-5	4
----	---	----	---	----	---	---	----	---

Cross Sum Calculation :

Pivot Element: -1

Left Sub Sum = Max (-1, -1+4, -1+4-3, -1+4-3+1, -1+4-3+1-2)
 = Max (-1, 3, 0, 1, -2)
 = 3

Right Sub Sum = Max (2, 2+1, 2+1-5, 2+1-5+4)
 = Max (2, 3, -2, 2)
 = 3

Cross Sum = Left Sub Sum + Right Sub Sum = (3+3) = 6

Cross Sum = 6

Maximum Sum = Max (Left Sum, Right Sum, Cross Sum) = 6



Maximum-Subarray Problem- Divide and Conquer Approach- Demonstration

Input : -2, 1, -3, 4, -1, 2, 1, -5, 4

Left Sum = 4, Right Sum = 4,
Cross Sum = ??

-2	1	-3	4	-1	2	1	-5	4
----	---	----	---	----	---	---	----	---

Cross Sum Calculation :

Pivot Element: -1

$$\begin{aligned}\text{Left Sub Sum} &= \text{Max} (-1, -1+4, -1+4-3, -1+4-3+1, -1+4-3+1-2) \\ &= \text{Max} (-1, 3, 0, 1, -2) \\ &= 3\end{aligned}$$

$$\begin{aligned}\text{Right Sub Sum} &= \text{Max} (2, 2+1, 2+1-5, 2+1-5+4) \\ &= \text{Max} (2, 3, -2, 2) \\ &= 3\end{aligned}$$

$$\text{Cross Sum} = \text{Left Sub Sum} + \text{Right Sub Sum} = (3+3) = 6$$

Cross Sum = 6

Answer : 6

Maximum Sum = Max (Left Sum, Right Sum, Cross Sum) = 6



FIND-MAXIMUM-SUBARRAY(*A*, *low*, *high*)

```
1  if high == low
2      return (low, high, A[low]) // base case: only one element
3  else mid =  $\lfloor (\textit{low} + \textit{high}) / 2 \rfloor$ 
4      (left-low, left-high, left-sum) =
          FIND-MAXIMUM-SUBARRAY(A, low, mid)
5      (right-low, right-high, right-sum) =
          FIND-MAXIMUM-SUBARRAY(A, mid + 1, high)
6      (cross-low, cross-high, cross-sum) =
          FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
7      if left-sum  $\geq$  right-sum and left-sum  $\geq$  cross-sum
8          return (left-low, left-high, left-sum)
9      elseif right-sum  $\geq$  left-sum and right-sum  $\geq$  cross-sum
10         return (right-low, right-high, right-sum)
11     else return (cross-low, cross-high, cross-sum)
```

FIND-MAX-CROSSING-SUBARRAY ($A, low, mid, high$)

```
1  left-sum =  $-\infty$ 
2  sum = 0
3  for  $i = mid$  downto  $low$ 
4      sum = sum +  $A[i]$ 
5      if sum > left-sum
6          left-sum = sum
7          max-left =  $i$ 
8  right-sum =  $-\infty$ 
9  sum = 0
10 for  $j = mid + 1$  to  $high$ 
11     sum = sum +  $A[j]$ 
12     if sum > right-sum
13         right-sum = sum
14         max-right =  $j$ 
15 return (max-left, max-right, left-sum + right-sum)
```

Maximum-Subarray Problem- Divide and Conquer Approach- Time Analysis

- **Time Complexity:** `maxSubArraySum()` is a recursive method and time complexity can be expressed as following recurrence relation. $T(n) = 2T(n/2) + \Theta(n)$ - By Applying Master Theorem we can get:
- **Time Complexity :** $O(n \log n)$





Thank you